Unit -1

Software engineering

Chapter-1

Introduction

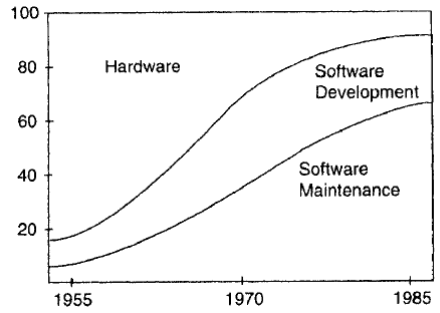
IEEE defines software as the collection of computer programs, procedures, rules and associated documents and data. This definition clearly states that software is not just programs, but includes all the associated documents.

The Problem Domain:

I) Industrial Strength Software:

A student system is primarily meant for demonstration purposes it is generally not used for solving any real problem of any organization.

II) Software is expensive: With the advanced technology, the cost of the hardware consistently decreasing. On the other hand the cost of the software is increasing. As a result, the HW/SW ratio for a computer system has shown a reversal from the early years as shown in below fig:



The main reason for the high cost of software is that software development is labor –intensive. As the main cost of producing software is the manpower employed, the cost of developing software is generally measured in terms of person-months of effort spent in development. And productivity is frequently measured in the industry in terms of DLOC (Delivered Line of Code) per person-month. To get an idea of the costs involved, let us consider the current state of practice in the industry. Lines of code (LOC) or thousands of lines of code (KLOC) delivered is by far the most commonly used measure of software size in the industry.

III) Late, Costly and Unreliable: A software industry has gained a reputation of not being able to deliver on time and within a budget. In a survey of over 600 firms, more than 35% reported having some computer-related development project that they categorized as a runaway. A runaway is not a project that is somewhat late or somewhat over budget—it is one where the budget and schedule are out of control.

There are many reasons of software unreliability. The main cause of unreliability in software is software failures. These failures are different from failures of electrical and machine failure. In software, the failures occur due to bugs or errors that get introduced during design and development process.

IV) Problem of Change and Rework: Once the software is delivered and deployed, it enters the maintenance phase. Software needs to be maintained not because some of its components

wear out and need to be replaced, but because there are often some residual errors remaining in the system that must be removed as they are discovered. These errors once discovered, need to be removed, leading to the software being changed. This is sometimes called corrective maintenance. Even without bugs, software frequently undergoes change. The main reason is that software often must be upgraded and enhanced to include more features and provide more services. This also requires modification of the software. The changed software then changes the environment, which in turn requires further change. This phenomenon is sometimes called the law of software evolution. Maintenance due to this phenomenon is sometimes called adaptive maintenance.

Maintenance involves understanding the existing software (code and related documents), understanding the effects of change, making the changes—to both the code and the documents—testing the new parts, and retesting the old parts that were not changed. Because often during development, the needs of the maintainers are not kept in mind, few support documents are produced during development to help the maintainer. As time goes by and the understanding of the system improves, the clients frequently discover additional requirements they had not specified earlier. This leads to requirements getting changed. This change leads to rework, the requirements, the design, the code all have to be changed to accommodate the new or changed requirements.

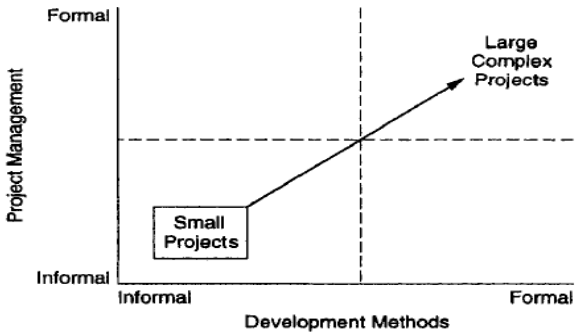
Definition

Software engineering is defined as the systematic approach to the development, operation, maintenance, and retirement of software.

The problems of software engineering

a) The problem of scale: The fundamental problem that software engineering must deal with is the issue of scale; development of a very large system requires a very different set of methods compared to developing a small system. Methods that one can use to develop programs of a few 100 lines cannot be expected to work when software of a few 1000 lines needs to be developed.

A different set of methods must be used for developing large software. Any large project involves the use of technology and project management. For software projects, by technology we mean the methods, procedures, and tools that are used. In small projects, informal methods for development and management can be used. However, for large projects, both have to be much more formal, as shown in fig.



b) Cost, Schedule & Quality:

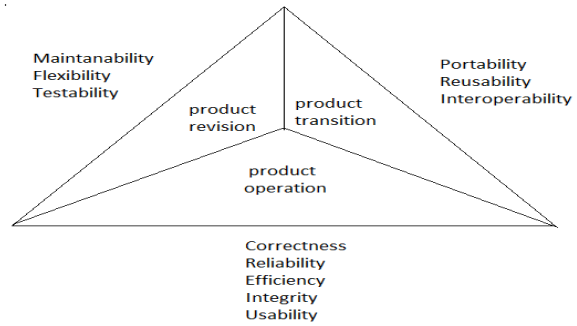
The Goal of any software engineering discipline is to develop high quality software with low cost. The cost of developing a system is the cost of the resources used for the system, which, in the case of software, is dominated by the manpower cost, as development is largely labor-intensive. Hence, the cost of a software project is often measured in terms of person-months, i.e., the cost is considered to be the total number of person-months spent in the project.

Schedule is another important factor in software engineering. Business trends are dictating that the time to market of a product should be reduced; that is, the cycle time from concept to delivery should be small. For software this means that it needs to be developed faster. Hence, any software product should be completed within a small cycle time. In order to reduce the cycle time, we can use RAD (Rapid Application Development) type models.

Quality:

The major factor driving any production discipline is quality. Today, quality is a main mantra, and business strategies are designed around quality. Clearly, developing high-quality software is another fundamental goal of software engineering. Quality of a software product as having three dimensions. The three dimensions are shown below

- Product operation
- Product transition
- Product revision



Product operations, deals with quality factors such as correctness, reliability, efficiency, integrity and usability. Product transition deals with quality factors like portability, reusability and interoperability. Product revision deals with maintainability, flexibility and testability.

- Correctness is the extent to which a program satisfies its specification.
- Reliability is the property that how well the software meets its requirements.

- Integrity. Extent to which access to software or data by unauthorized persons can be controlled.
- Efficiency is a factor in all issue relating to the execution of software. It includes response time, memory requirement and throughput.
- Usability is an important property that emphasizes the human aspects of the system.
- Maintainability is the effort required to locate and fix the errors in operating programs.
- Testability is the effort required to test and to ensure that system or a module performs its intended function.
- Flexibility is the effort required to modify an operating program or to enhance the functionality.
- Portability is the effort required to transfer the software from one hardware to another.
- Reusability is the extent to which parts of the software can be reused in other related applications.
- Interoperability is the effort required to couple the system with other system.

c) The problem of consistency:

A key challenge that software engineering faces is how to ensure that successful results can be repeated, and there can be some degree of consistency in quality and productivity.

An organization involved in software development not only wants high quality and productivity, but it wants these consistently. In other words, a software development organization would like to produce consistent quality software with consistent productivity.

Consistency of performance is an important factor for any organization; it allows an organization to predict the outcome of a project with reasonable accuracy, and to improve its processes to produce higher-quality products and to improve its productivity.

The software engineering approach

Phased development process: A development process consists of various phases, each phase ending with a defined output. The phases are performed in an order specified by the process model being followed. The main reason for having a phased process is that it breaks the problem of developing software into successfully performing a set of phases, each handling a different concern of software development. A phased process allows proper checking for quality and progress at some defined points during the development.

I) Requirement Analysis

Requirements analysis is done in order to understand the problem the software system is to solve. The emphasis in requirements analysis is on identifying what is needed from the system, not how the system will achieve its goals. In most software projects this phase ends with a document describing all the requirements.

The goal of the requirements activity is to document the requirements in a software requirements specification document (SRS). This phase is done by an analyst.

There are two major activities in this phase: problem analysis and requirement specification. In problem analysis, the aim is to understand the problem and its context, and the requirements of the new system that is to be developed.

Once the problem is analyzed and the essentials understood, the requirements must be specified in the requirement specification document. The requirements document must specify all functional and performance requirements; the formats of inputs and outputs; and all design constraints etc.

II) Software design

The purpose of the design phase is to plan a solution of the problem specified by the SRS. This phase is the first step in moving from the problem domain to the solution domain. In other words, starting with what is needed; design takes us toward how to satisfy the needs. The output of this phase is design documents.

There are 2 design activities i) system design and ii) detailed design The system design identifies the modules that should be built for developing the system and the specifications of these modules. At the end of system design all the major data structures, file formats, output formats, etc., are also fixed. In detailed design, the internal logic of each of the modules is specified.

III) Coding

The goal of the coding phase is to translate the design of the system into code in a given programming language. For a given design, the aim in this phase is to implement the design in the best possible manner. The coding phase affects both testing and maintenance profoundly. Well written code can reduce the testing and maintenance effort.

IV) Testing

Testing is the major quality control measure used during software development. Its basic function is to detect defects in the software. During requirements analysis and design, the output is a document that is usually textual and non-executable. After coding, computer programs are available that can be executed for testing purposes.

The goal of testing is to uncover requirement, design, and coding errors in the programs. The starting point of testing is unit testing, where the different modules or components are tested individually. As modules are integrated into the system, integration testing is performed, which focuses on testing the interconnection between modules. After the system is put together, system testing is performed. Here the system is tested against the system requirements to see if all the requirements are met and if the system performs as specified by the requirements. Finally, acceptance testing is performed to demonstrate to the client, on the real data of the client, the operation of the system.

Testing is an extremely critical and time-consuming activity. It requires proper planning of the overall testing process. Frequently the testing process starts with a *test plan* that identifies all the testing-related activities that must be performed and specifies the schedule, allocates the resources, and specifies guidelines for testing. The test plan specifies conditions that should be tested, different units to be tested, and the manner in which the modules will be integrated. Then for different test units, a *test case* specification document is produced, which lists all the different test cases, together with the expected outputs. During the testing of the unit, the specified test cases are executed and the actual result compared with the expected output. The final output of the testing phase is the *test*

report and the error report, or a set of such reports. Each test report contains the set of test cases and the result of executing the code with these test cases. The error report describes the errors encountered and the action taken to remove the errors.

Chapter-2

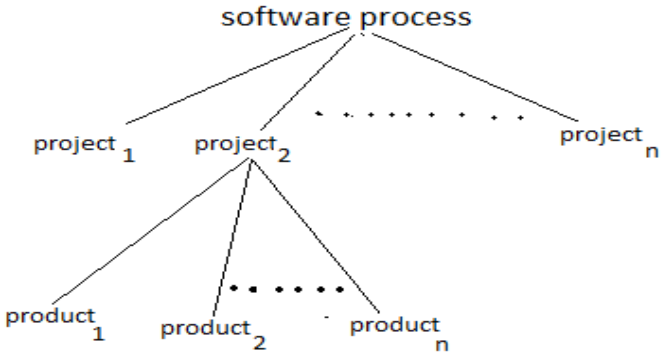
Software Processes

A software process is a set of activities, together with ordering constraints among them, such that if the activities are performed properly and in accordance with the ordering constraints, the desired result is produced. The basic desired result is, as stated earlier, high quality at low cost.

The process that deals with the technical and management issues of software development is called a software process; clearly many different activities need to be performed to develop software.

Process, Projects and Products:

A software process specifies the method of developing software. A software project is a development project in which a software process is used. Software products are outcomes of a software project. Each software development project starts with some needs and ends with some software that satisfies those needs. One can view a software process is an abstract type and one process can have many projects and a project can produce many products.



Components of software Processes

The 3 types of entities that a software engineering deals with are processes, projects and products. Products require different processes. The major process dealing with products is the development process responsible for producing the desired products and other products.

The basic goal of this process is to develop a product that will satisfy the customer. A software project is clearly a dynamic entity in which activities are performed, and project management process needed to properly control this dynamic activity. For large projects, project management is more important than technical methods for success of the project.

There are 2 major components in a software process- a development process and a project management process. The development process and project management process aim to satisfy the cost and quality objectives of the project.

The development process generally cannot handle change request at an arbitrary point of time, to handle the inevitable change and rework requests another process called Software Configuration Control Process is used. The objective of this components process is to deal with managing change, so that cost and quality are met and integrity of the product is not violated despite these change requests.

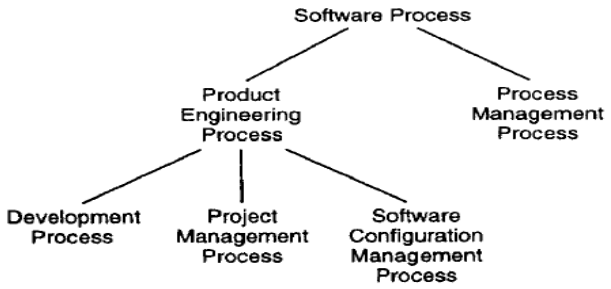
Overall particular projects, to satisfy the objectives and handle the realities of software, at least three major processes are needed:

i) Development Process
ii) Project Management Process
iii) Configuration control process.

These three processes focus on projects and products. They can be all considered as Product Engineering Processes as their main objective is to produce desired product.

The process management process deals with the software process; its basic objective is to improve the software process. By improvement, we mean that the capability of the process to produce quality goods at low cost is improved. For this, the current software process is studied, frequently by studying the projects that have been done using the process.

The relationship between these major component processes is shown in fig:



Software processes.

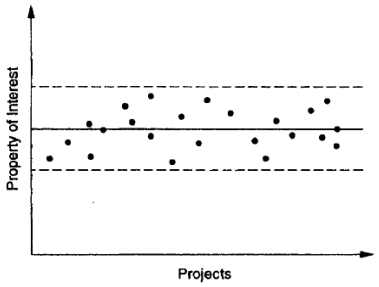
Characteristics of a software Processes

The basic objectives of a process are optimality and scalability. The Optimality means that the process should be able to produce a high-quality of software at low cost, and Scalability means that it should also applicable for large software projects. To achieve these objectives a process should have some properties,

Predictability: Predictability of a process is determines how accurately the outcome of following process in a project can be predicted before project is completed. Predictability can be considered a fundamental property of any process. In fact, if a process is not predictable, it is of limited use. Effective project plan is essential for success of a project, and project management revolves around the project plan. S project plan contains cost and schedule estimates for the project, along with plans for quality assurance and other activities. Any estimation about a project is based on the properties of the project, and capacity or past experience of the organization.

The fundamental basis for quality prediction is that quality of the product is determined largely by the process used to develop it. Using this basis, quality of the product of a project can be estimated or predicted by seeing the quality of the products that have been produced in the past by the process being used in the current project. In fact, effective management of quality control activities largely depends on the predictability of the process. For example, for effective quality control, one method is to estimate what types and quantity of errors will be detected at what stage of the development, and then use them to determine if the quality assurance activities are being performed properly. This can only be done if the process is predictable; based on the past experience of such a process one can estimate the distribution of errors for the current project.

A predictable process is also said to be under statistical control. A process is under statistical control if following the same process produces similar results—results will have some variation, but the variation is mostly due to random causes and not due to process issues. This is shown in Fig



The y-axis represents some property of interest (quality, productivity, etc.), and x-axis represents the projects. The dark line is the expected value of the property for this process. Statistical control implies that most of the times the property of interest will be within a bound around the expected value.

It should be clear that if one hopes to consistently develop software of high quality at low cost, it is necessary to have a process that is under statistical control. A predictable process is an essential requirement for ensuring good quality and low cost.

Support testability and maintainability

In the life of software the maintenance costs generally exceed the development costs. Clearly, if we want to reduce the overall cost of software or achieve "global" optimality in terms of cost rather than "local" optimality in terms of development cost only, the goal of development should be to reduce the maintenance effort.

A process generally includes requirements, design, coding, and testing phases. Of the development cost, an example distribution of effort with the different phases could be:

Requirements	10%
Design	10%
Coding	30%
Testing	50%

Effort distribution with phases

Another way of determining the effort spent in programming is to study how programmer spent their time in a software organization. The data are found in below table;

Wring programs	13%
Reading program manuals	16%
Job Communication	32%
Others (including Personal)	39%

This data clearly shows that programming is not the major activity on which programmers spend their time. Even if we take away the time spent in "other" activities, the time spent by a programmer writing programs is still less than 25% of the remaining time. It was found that programmers spend less than 20% of their time programming.

The second important observation from the data about effort distribution with phases is that testing consumes the most resources during development. This is, again, contrary to the common practice, which considers testing a side activity that is often not properly planned.

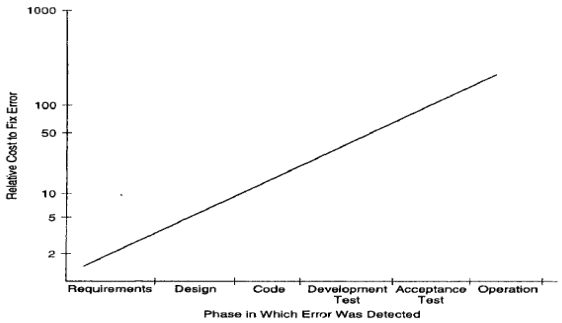
The goal of the process should not be to reduce the effort of design and coding, but to reduce the cost of testing and maintenance. Both testing and maintenance depend heavily on the quality of design and code, and these costs can be considerably reduced if the software is designed and coded to make testing and maintenance easier.

Early Defect Removal and Defect Prevention

Errors can occur at any stage during development. An example distribution of error occurrences by phase is:

Requirements	20%
Design	30%
Coding	50%

As we can see, errors occur throughout the development process. However, the cost of correcting errors of different phases is not the same and depends on when the error is detected and corrected. The relative cost of correcting requirement errors as a function of where they are detected is shown below,



Error detection and correction should be a continuous process that is done throughout software development. In terms of development phases, this means that we should try to verify the output of each phase before starting with the next. In other words, a process should have quality control activities spread through the process and in each phase. A quality control (QC) activity is one whose main purpose is to identify and remove defects.

Having QC tasks through the development is clearly an objective that should be supported by the process. However, it is even better to provide support for defect prevention.

Process Improvement & feedback

Improving the quality and reducing the cost of products are fundamental goals of any engineering discipline. In the context of software, as the productivity and quality are determined largely by the process, to satisfy the objectives of quality improvement and cost reduction, the software process must be improved.

Having process improvement as a fundamental objective requires that the software process be a closed-loop process. That is, the process must be improved based on previous experiences, and each project done using the existing process must feed information back to facilitate this improvement. As stated earlier, this activity of analyzing and improving the process is largely done in the process management component of the software process.

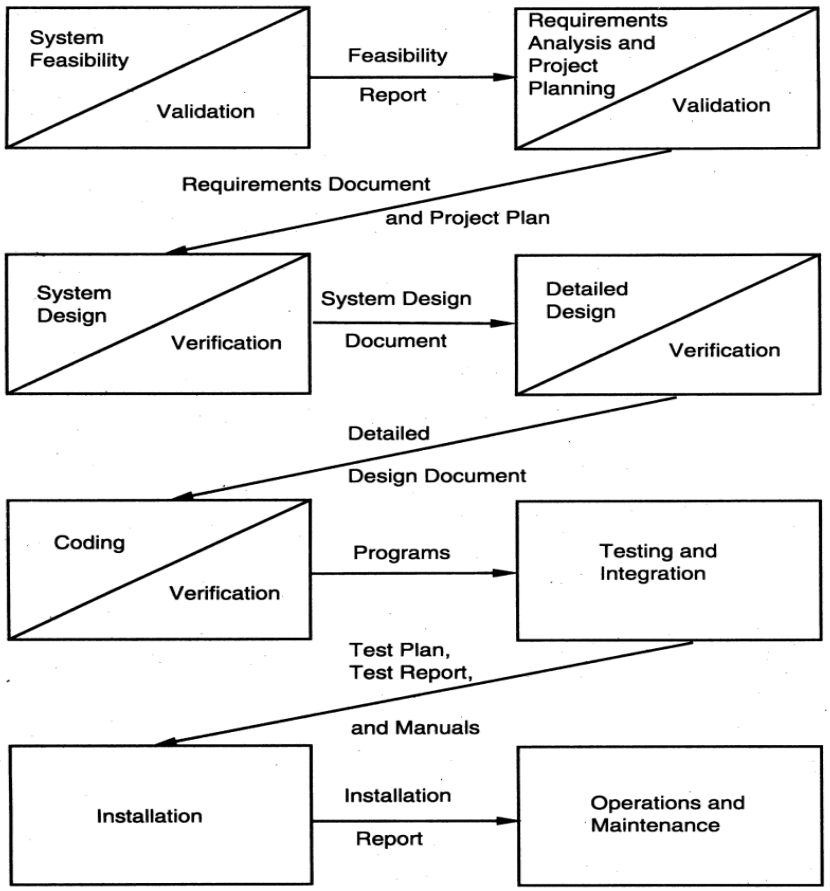
However, to support this activity, information from various other processes will have to flow to the process management process.

Software Development Process

In the software development process we focus on the activities directly related to production of the software, for example, design, coding, and testing. As the development process specifies the major development and quality control activities that need to be performed in the project, the development process really forms the core of the software process. The management process is decided based on the development process. Due to the importance of the development process, various models have been proposed.

Waterfall Model:

The simplest process model is the waterfall model, which states that the phases are organized in a linear order.



In this model, a project begins with feasibility analysis. Upon successfully demonstrating the feasibility of a project, the requirements analysis and project planning begins. The design starts after the requirements analysis is complete, and coding begins after the design is complete. Once the programming is completed, the code is integrated and testing is done. Upon successful completion of testing, the system is installed. After this, the regular operation and maintenance of the system takes place.

The requirements analysis phase is mentioned as "analysis and planning." Planning is a critical activity in software development. A good plan is based on the requirements of the system and should be

done before later phases begin. Planning usually overlaps with the requirements analysis, and a plan is ready before the later phases begin.

This plan is an additional input to all the later phases. Linear ordering of activities has some important consequences. First, to clearly identify the end of a phase and the beginning of the next, some certification mechanism has to be employed at the end of each phase. This is usually done by some verification and validation means that will ensure that the output of a phase is consistent with its input (which is the output of the previous phase), and that the output of the phase is consistent with the overall requirements of the system.

Verification: "Are we building the product right?"
Validation: "Are we building the right product?"

The outputs of the earlier phases are often called work products and are usually in the form of documents like the requirements document or design document. For the coding phase, the output is the code. Though the set of documents that should be produced in a project is dependent on how the process is implemented, the following documents generally form a reasonable set that should be produced in each project:

- Requirements document
- Project plan
- Design documents (architecture, system, detailed)
- Test plan and test reports
- Final code
- Software manuals (e.g., user, installation, etc.)

In addition to these work products, there are various other documents that are produced in a typical project. These include review reports, which are the outcome of reviews conducted for work products, as well as status reports that summarize the status of the project on a regular basis.

Advantages

- This model is simple.
- It is conceptually straightforward and divides the large task of building a software system into a series of cleanly divided phases, each phase dealing with a separate logical concern.
- It is also easy to administer in a contractual setup. Initiative and logical.

Weakness

- Model uses “all or nothing approach”
- Requirements frozen easily
- Model may choose outdated hardware technology
- User feedback are not allowed in this model

Limitations of Waterfall model (disadvantages) :

1. It assumes that the requirements of a system can be frozen (i.e., baselined) before the design begins. This is possible for systems designed to automate an existing manual system. But for new systems, determining the requirements is difficult as the user does not even know the requirements. Hence, having unchanging requirements is unrealistic for such projects.

2. Freezing the requirements usually requires choosing the hardware. A large project might take a few years to complete. If the hardware is selected early, then due to the speed at which hardware technology is changing, it is likely that the final software will use a hardware technology on the verge of becoming obsolete. This is clearly not desirable for such expensive software systems.

3. It follows the "big bang" approach—the entire software is delivered in one shot at the end. This entails heavy risks, as the user does not know until the very end what they are getting. Furthermore, if the project runs out of money in the middle, then there will be no software.

4. It is a document-driven process that requires formal documents at the end of each phase.

The waterfall model has been the most widely used process model. It is well suited for routine types of projects where the requirements are well understood. That is, if the developing organization is quite familiar with the problem domain and the requirements for the software are quite clear, the waterfall model works well.

Prototyping Model

The goal of a prototyping-based development process is to counter the first two limitations of the waterfall model.

The basic idea here is that instead of freezing the requirements before any design or coding can proceed, a throwaway prototype is built to help understand the requirements.

This prototype is developed based on the currently known requirements.

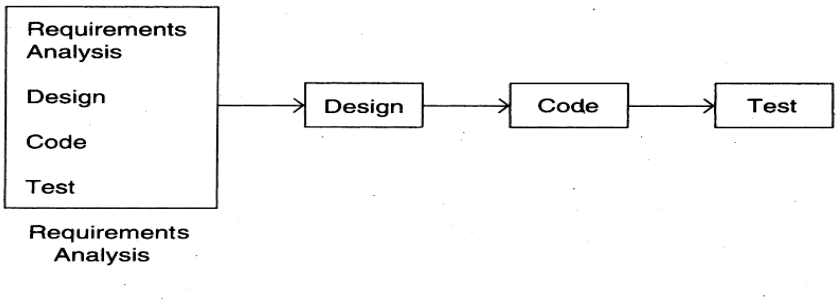
Development of the prototype obviously undergoes design, coding, and testing, but each of these phases is not done very formally or thoroughly.

By using this prototype, the client can get an actual feel of the system; because the interactions with the prototype can enable the client to, better understand the requirements of the desired system. This results in more stable requirements that change less frequently.

Prototyping is an attractive idea for complicated and large systems for which there is no manual process or existing system to help determine the requirements. In such situations, letting the client "play/use" with the prototype provides invaluable and intangible inputs that help determine the requirements for the system.

It is also an effective method of demonstrating the feasibility of a certain approach. This might be needed for novel systems, where it is not clear those constraints can be met or that algorithms can

be developed to implement the requirements. In both situations, the risks associated with the projects are being reduced through the use of prototyping. The process model of the prototyping approach is shown in fig:



Prototype model

A development process using throwaway prototyping typically proceeds as follows

The development of the prototype typically starts when the preliminary version of the requirements specification document has been developed. At this stage, there is a reasonable understanding of the system and its needs and which needs are unclear or likely to change.

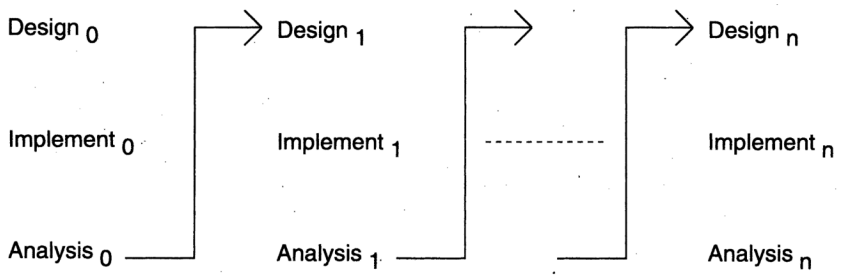
After the prototype has been developed, the end users and clients are given an opportunity to use and explore the prototype. Based on their experience, they provide feedback to the developers regarding the prototype: what is correct, what needs to be modified, what is missing, what is not needed, etc. Based on the feedback, the prototype is modified to incorporate some of the suggested changes that can be done easily, and then the users and the clients are again allowed to use the system. This cycle repeats until, in the judgment of the prototype developers and analysts, the benefit from further changing the system and obtaining feedback is outweighed by the cost and time involved in making the changes and obtaining the feedback. Based on the feedback, the initial requirements are modified to produce the final requirements specification, which is then used to develop the production quality system.

Prototyping is well suited for projects where requirements are hard to determine and the confidence in the stated requirements is low. In such projects where requirements are not properly understood in the beginning, using the prototyping process model can be the most effective method for developing the software. It is also an excellent technique for reducing some types of risks associated with a project.

Iterative Enhancement

The iterative development process model counters the third and fourth limitations of the waterfall model and tries to combine the benefits of both prototyping and the waterfall model. The basic idea is that the software should be developed in increments, each increment adding some functional capability to the system until the full system is implemented.

The iterative enhancement model is shown in Fig



In the first step of this model, a simple initial implementation is done for a subset of the overall problem.

This subset is one that contains some of the key aspects of the problem that are easy to understand and implement and which form a useful and usable system.

A project control list is created that contains all the tasks that must be performed to obtain the final implementation. This project control list gives an idea of how far along the project is at any given step from the final system.

Each step consists of removing the next task from the list, designing the implementation for the selected task, coding and testing the implementation, performing an analysis of the partial system obtained after this step, and updating the list as a result of the analysis.

These three phases are called the design phase, implementation phase, and analysis phase. The process is iterated until the project control list is empty, at which time the final implementation of the system will be available.

Advantage:

- Develop high-risk or major functions first
- Each release delivers an operational product
- Customer can respond to each build
- Uses “divide and conquer” breakdown of tasks
- Lowers initial delivery cost
- Initial product delivery is faster
- Customers get important functionality early
- Risk of changing requirements is reduced

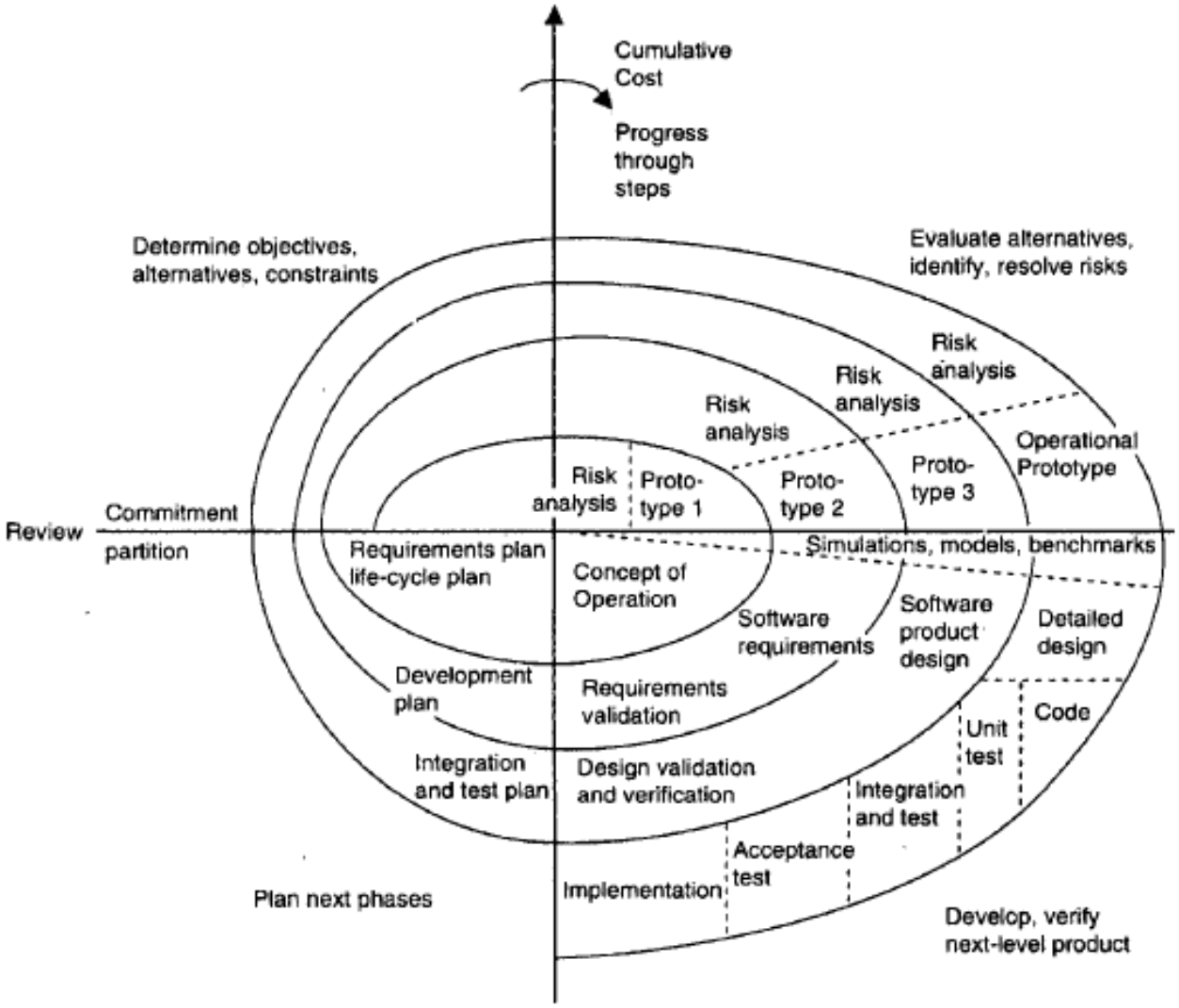
Weakness:

- Requires good planning and design

- Requires early definition of a complete and fully functional system to allow for the definition of increments
- Well-defined module interfaces are required (some will be developed long before others)
- Total cost of the complete system is not lower

Spiral Model

The spiral model is another important model proposed for software development process. As the name suggests, the activities in this model can be organized like a spiral that has many cycles as shown in Fig



Each cycle in the spiral begins with the identification of objectives for that cycle, the different alternatives that are possible for achieving the objectives, and the constraints that exist. The next step in the cycle is to evaluate these different alternatives based on the objectives and constraints.
The focus of evaluation in this step is based on the risk perception for the project. The next step is to develop strategies that resolve the uncertainties and risks. This step may involve activities such as benchmarking, simulation, and prototyping. Next, the software is developed, keeping in mind the risks. Finally the next stage is planned.

The spiral model is similar to the incremental model, with more emphases placed on risk analysis. The spiral model has four phases: Planning, Risk Analysis, Engineering and Evaluation. A software project repeatedly passes through these phases in iterations (called Spirals in this model). The baseline spiral, starting in the planning phase, requirements are gathered and risk is assessed. Each subsequent spiral builds on the baseline spiral. Requirements are gathered during the planning phase. In the risk analysis phase, a process is undertaken to identify risk and alternate solutions.
A prototype is produced at the end of the risk analysis phase. Software is produced in the engineering phase, along with testing at the end of the phase. The evaluation phase allows the customer to evaluate the output of the project to date before the project continues to the next spiral. In the spiral model, the angular component represents progress, and the radius of the spiral represents cost.

Advantages

1. High amount of risk analysis.
2. Good for large and mission-critical projects.
3. Software is produced early in the software life cycle.

Disadvantages

1. Can be a costly model to use.
2. Risk analysis requires highly specific expertise.
3. Project’s success is highly dependent on the risk analysis phase.
4. Doesn’t work well for smaller projects.

Project Management Processes

Proper management is an integral part of software development. A large software development project involves many people working for a long period of time. We have seen that a development process typically partitions the problem of developing software into a set of phases.

To meet the cost, quality, and schedule objectives, resources have to be properly allocated to each activity for the project, and progress of different activities has to be monitored and corrective actions taken, if needed. All these activities are part of the project management process.

The focus is on issues like planning a project, estimating resource and schedule, and monitoring and controlling the project.

Phases of Project Management Process:

The activities in the management process for a project can be grouped broadly into three phases: planning, monitoring and control, and termination analysis.

Project management begins with planning, which is perhaps the most critical project management activity. The goal of this phase is to develop a plan for software development following which the objectives of the project can be met successfully and efficiently. A software plan is usually produced before the development activity begins and is updated as development proceeds and data about progress of the project becomes available. During planning, the major activities are cost estimation, schedule and milestone determination, project staffing, quality control plans, and controlling and monitoring plans. Project planning is undoubtedly the single most important management activity, and it forms the basis for monitoring and control. We will devote one full chapter later in the book to project planning.

Project monitoring and control phase of the management process is the longest in terms of duration; it encompasses most of the development process. It includes all activities the project management has to perform while the development is going on to ensure that project objectives are met and the development proceeds according to the developed plan.

As cost, schedule, and quality are the major driving forces, most of the activity of this phase revolves around monitoring factors that affect these. Monitoring potential risks for the project, which might prevent the project from meeting its objectives, is another important activity during this phase.

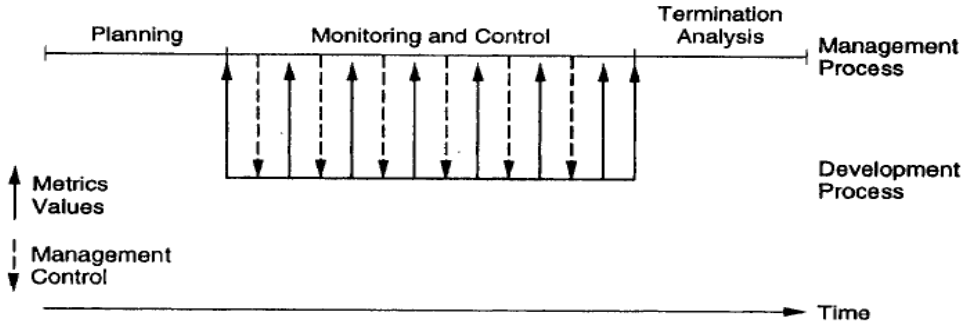


Fig: Temporal relationship between development and management process.

Monitoring a development process requires proper information about the project. Such information is typically obtained by the management process from the development process. Monitoring and control lasts the entire duration of the project, the last phase of the management process—termination analysis—is performed when the development process is over. The basic reason for performing termination analysis is to provide information about the development process and learn from the project in order to improve the process. This phase is also often called postmortem analysis. In iterative development, this analysis can be done after each iteration to provide feedback to improve the execution of further iterations.

Metrics, Measurements and Models

For effective project monitoring, the information coming from the development process to the management process should be objective and quantitative data about the project.

Software metrics are quantifiable measure that could be used to measure different characteristics of a software system or the software development process.

A number of metrics have been proposed to quantify thing like size, complexity and reliability of software.

The measurement method must be objective and should produce the same result independent of measurer.

A model is a relationship of the predicted variables that other variable that other variables that can be measured. If there is metric of interest which cannot be measured directly, then we build a model to estimate the metric value based on the value of some other metrics that we can measure.

The model may be determined based on empirical data and or it may be analytic data.

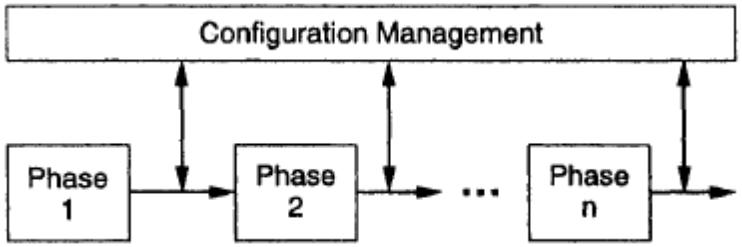
Software Configuration Control Management Process: (SCM)

Configuration management (CM) or software configuration management (SCM) is the discipline for systematically controlling the changes that take place during development.

The IEEE defines SCM as "the process of identifying and defining the items in the system, controlling the change of these items throughout their life cycle, recording and reporting the status of items and change requests, and verifying the completeness and correctness of items"

Development process handles normal changes such as change in code while the programmer is developing it, changes in requirements during the requirement phase etc...

Software configuration management is a process independent of the development process largely because most development models accommodate changes at any time during the development, So that changes are allowed in a controlled manner, as shown in Fig



The basic reason for having SCM, as with any other activities for a project, is that it has beneficial effects on cost, schedule, and quality of the product being developed.
The fundamental reason for SCM process in to control the changes so that they have minimal effect on cost, schedule and quality.

SCM can be considered as having three major components:

- Software Configuration Identification
- Change Control
- Status Accounting and Auditing.

Software Configuration Identification

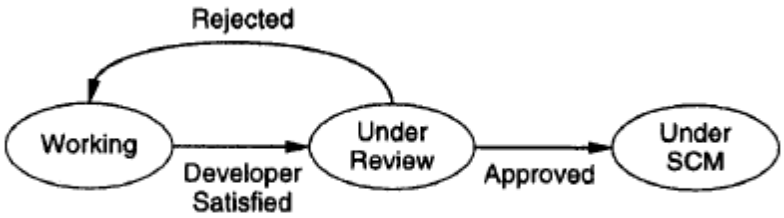
The first requirement for any change management is to a have a clearly agreed-on basic change.
As baseline change is the changing of the established baseline, which is controlled by SCM.
A baseline is also forms a reference point in the development of a system. At the time it is established, a software baseline represents the software in the most recent state.
Some of the common baselines are Functional or requirement baseline, design baseline, and product or system baseline.
Functional baseline is generally functional requirements for the software.
Design baseline consists of the different components in the software and their design.
Product baseline represents the developed system.
The baseline is developed only after the product is stable.
A baseline can consists of many software configuration items (SCI) or items. A baseline is an arrangement of a set of SCIs and relationship between them.

The SCIs are the basic unit for change control. The SCIs being managed by SCM are not isolated there is a dependency between various SCIs.

Change Control

Once the SCIs are identified and their dependencies are understood, the change control procedures of the SCM can be applied. Most of the decisions regarding changes are generally taken by the Configuration Control Board (CCB) which is a group of people responsible for configuration management headed by a Configuration Manager (CM) .For small project the CCB might consists of just one person.

While an SCI is under development then it is considered as Working State. An SCI in working state is not in SCM, it can be changed freely. Once the SCIs are stable it is used by others, the SCI is given to CM for review. And items are entered in to under review status. The CM reviews the SCIs and if it is approved, it enters into a library state, after which the item is formally under SCM.



SCM life cycle of a product

Once an SCI is in the library. It cannot be modified, even by the author/ developer without the permission of the CM. An SCI under SCM can be changed only if the change has been approved by the CM. In such case the SCI is checked out off the library, the change made to the SCI and then modified SCI is given back to the client. This aspect of SCM is called library management.

The changed SCI do not replace the Old copy, but a new version of the SCI is created. With multiple versions of SCI and multiple versions of system using different versions of each SCI, Version Management becomes extremely important.

A change is initiated by a change request (CR) .the reason for changing can be anything. The most common reasons are requirement changes, changes due to bugs, platform changes and enhancement changes.

The CR for change generally consists of three parts. The first part describes change, the reasons for change and SCI that are affected. The second part filled by CM describes the decisions taken by CCB on this CR (approved on not approved), the actions the CM feels need to be done to implement this change and comments by CM. The third part is filled by implementor who later implements the change. The CR is assigned by a unique number by CCB, which is used to refer to it. A CR will contain the following information

CR ID
About the change Request
Items to be changed
Description of change
Reasons for change
Priority of the change
Originator
CCB Comments
CR approved/ rejected
Actions needed
Comments
Implementation information
Status of change implementation
Comments

One of the most common reasons for CR is the discovery of some bugs or problem in the software. Frequently, the change request originating due to faults are made on a different form called fault reports Fault reports are generally treated as high priority CRs. An FR is also assigned a unique ID by CM. An FR form generally contain

FR ID
Fault Information
Description of the Fault
Severity of the Fault
Items suspected of being faulty
Effects of the Fault
Circumstances and Environment data that caused the fault to manifest itself
Possible fixes
Originator
CCB comments
Approved /rejected
Priority
Comments
Fault fixing information
Items changed
Comments

Some changes might affect the baseline. If the CR affects the baseline then baseline has to be updated.

Status Accounting and Auditing

Incorporating changes, after they have been approved takes time. So the changes to the items (SCIs) over a period of time. Some methods are needed to record the current status of the CR/FR. This can be used by the manager to determine the status of the entire change request.
The aim of status accounting is to answer the questions like status of CR? Approved or Not? Status of an Approval? Frequency of CRs? What is average time and fix the CR? Want are the number of CRs per SCI? etc.
For status accounting the main source of information is CRs and FRs.
Auditing has a different role in SCM. An audit for a process is performed by auditors. The main objective of an audit is to determine if the specified process satisfy the goals of the process. The same is done for SCM auditing
Records of past changes are evaluated by the auditors to determine if the SCM procedures are being followed, and the procedures are evaluated to ensure that the SCM goals are met.

Process Management Process

A software process is not a static entity—it has to change to improve so that the products produced using the process are of higher quality and are less costly. As we have seen, improving quality and productivity are fundamental goals of engineering. To achieve these goals the software process must continually be improved, as quality and productivity are determined to a great extent by the process.

In process management the focus is on improving the process which in turn improves the general quality and productivity for the products produced using the process. In project management the focus is on executing the current project and ensuring that the objectives of the project are met.
To improve its software process, an organization needs to first understand the status of the current status and then develop a plan to improve the process.

Once some process improvement takes place, the process state may change, and a new set of possibilities may emerge. This concept of introducing changes in small increments based on the current state of the process has been captured in the Capability Maturity Model (CMM) framework.

Buliding estimate models:

For define metrics to quantify many quality attributes, it is important to build models. These models are used for project mangement to predict various parameters of interest for the project.

A **Model** for a software process can be represented as

$Y = f(x1, x2, x3, \ldots Xn)$.

The dependent veriable y is the metric of interest(total effort , reliability etc..).
x1,x2,x3,…Xn are independent variables that typically represent some metric values that can be measured when this model is applied. The function f is really the model itself that specifies how Y depends on the independent variables for the process.

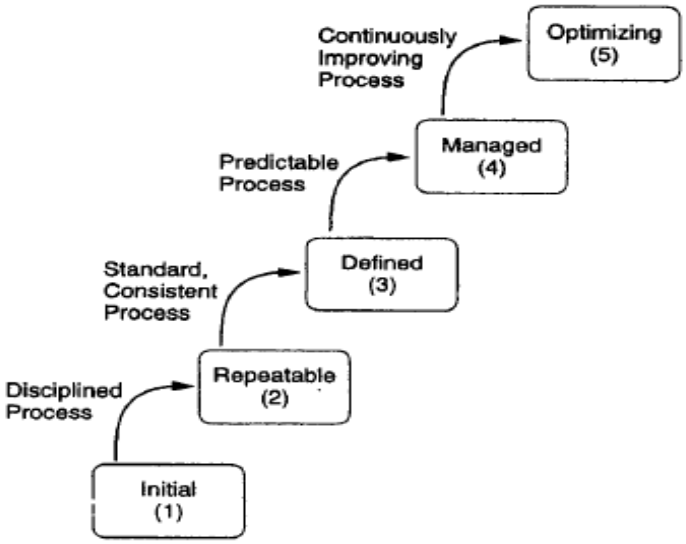
Capability Maturity Model

To improve its software process, an organization needs to first understand the status of the current status and then develop a plan to improve the process. The CMM framework provides a general roadmap for process improvement.

The process capability of an organization determines what can be expected from the organization in terms of quality and productivity. The goal of process improvement is to improve the process capability. A maturity level is a well-defined evolutionary idea towards achieving a mature software process. Based on the empirical evidence found by examining the processes of many organizations, the CMM suggests that there are five well-defined maturity levels for a software process.

These are initial (level 1), repeatable, defined, managed, and optimizing (level 5). The CMM framework says that as process improvement is best incorporated in small increments, processes go from their current levels to the next higher level when they are improved. Hence, during the course of process improvement, a process moves from level to level until it reaches level 5.

This is shown in Fig



Capability Maturity Model

The CMM provides characteristics of each level, which can be used to assess the current level of the process of an organization. As the movement from one level is to the next level, the characteristics of the levels also suggest the areas in which the process should be improved so that it can move to the next higher level.

The initial process (level 1) is essentially an ad hoc process that has no formalized method for any activity. Basic project controls for ensuring that activities are being done properly, and that the project plan is being adhered to, are missing. The process capability is unpredictable as the process constantly changes. Organizations at this level can benefit most by improving project management, quality assurance, and change control.

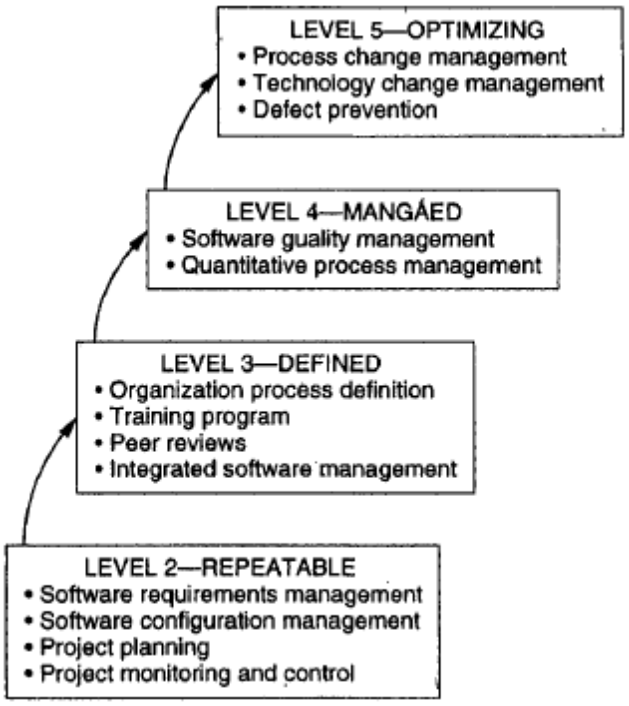
In a repeatable process (level 2), policies for managing a software project and procedures to implement those policies exist. That is, project management is well developed in a process at this level. Some of the characteristics of a process at this level are: project commitments are realistic and based on past experience with similar projects, cost and schedule are tracked and problems resolved when they arise, formal configuration control mechanisms are in place, and software project standards are defined and followed. Essentially, results obtained by this process can be repeated as the project planning and tracking is formal. Essentially, results obtained by this process can be repeated as the project planning and tracking is formal.

At the defined level (level 3) the organization has standardized a software process, which is properly documented. A software process group exists in the organization that owns and manages the process. In the process each step is carefully defined with verifiable entry and exit criteria, methodologies for performing the step, and verification mechanisms for the output of the step. In this process both the development and management processes are formal.
At the managed level (level 4) quantitative goals exist for process and products. Data is collected from software processes, which is used to build models to characterize the process. Hence, measurement plays an important role in a process at this level. Due to the models built, the organization has a good insight of the process capability and its deficiencies. The results of using such a process can be predicted in quantitative terms.

At the optimizing level (level 5), the focus of the organization is on continuous process improvement. Data is collected and routinely analyzed to identify areas that can be strengthened to improve quality or productivity. New technologies and tools are introduced and their effects measured in an effort to improve the performance of the process. Best software engineering and management practices are used throughout the organization.

This CMM framework can be used to improve the process. Improvement requires first assessing the level of the current process. Based on the current level, the areas in which maximum benefits can be derived are known from the framework.

The complete CMM framework provides more details about which particular areas need to be strengthened to up the maturity framework this is generally done by specifying key process areas of each maturity level.
Some of the key process areas of the different levels are shown in diagram.



Key process areas.

Quality improvement paradigm and GQM

Quality Improvement Paradigm (QIP) is another approach for improving a process. QIP does not specify levels or what areas to focus on for improvement.
It gives general method for improving a process essentially depends on the organization to which the process belongs and its objectives.
The basic idea of this approach is to understand the current process, set objectives for improvements and then plan and execute the improvement actions.
The QIP consists of 6 basic steps:

- Characterize: understand the current process and the environment it operates in.
- Set Goals: based on the understanding of the process and environment and objectives of the organization, set quantifiable goals for performance improvements.
- Choose Process: based on the characteristics and goals choose the component processes that should be changed to meet goals
- Execute: execute the projects using the processes and provide feedback data.
- Analyze: analyze the data at the end of each project. Form analysis determine problem and make recommendations for improvements to be applied on future project.
- Package: based on the experience gained from many projects, define and formalize the changes to be made to process and expectation from the new processes.

An important component of the QIP is feedback cycle from the projects. For collecting proper data that will serve the purpose of processes improvement, the Goal/Question/Metrics (GQM) paradigm is frequently used.

The GQM paradigm suggests a general framework for collecting data from projects that can be used for a specific purpose.

In this approach there is no general set of metrics and organization goals are to specify.

GQM is a top-down approach that starts from the objectives and works its way down to metrics needed.